./images/elte_cimer_szines.eps-converted-to.pdf

Eötvös Loránd University

Faculty of Informatics

Dept. of Algorithms and Applications

A Data-Driven Machine Learning Platform for Urban Air Quality Prediction and Risk Assessment

Supervisor:
Szabó László Ferenc
Associate Professor and Chair

Author:
Musaev Kutmanbek
Computer Science BSc

Budapest, 2026

Contents

1 Introduction **4**

1.1 Motivation 4

1.2 Background and Related Work 6

1.2.1 Time Series Forecasting: Classical Approaches 6

1.2.2 Deep Learning Approaches 7

1.2.3 Tree-Based Ensemble Methods 8

1.2.4 Existing Air Quality Platforms and Applications 9

1.2.5 Positioning of This Work 10

1.3 Problem Statement 10

1.4 Objectives 11

1.5 Dataset Description 12

1.5.1 The London Air Quality Network 12

1.5.2 Measured Variables 13

1.5.3 Temporal Coverage and Resolution 13

1.5.4 Data Quality Considerations 14

1.6 Thesis Structure 15

2 User Documentation **16**

2.1 System Requirements 16

2.1.1 Hardware Requirements 16

2.1.2 Software Requirements 16

2.1.3 Browser Requirements 17

2.2 Installation and Configuration 17

2.2.1 Cloning the Repository 17

2.2.2 Setting Up the Virtual Environment 17

2.2.3 Installing Dependencies 17

2.2.4 Configuring Secrets 18

2.2.5 Database Initialization 18
2.2.6 Model Training 18
2.2.7 Running the Application 18
2.2.8 Production Deployment Considerations 19
2.3 Authentication System 19
2.3.1 Registration Flow 19
2.3.2 Password Requirements 20
2.3.3 Login Process 20
2.3.4 Session Persistence 20
2.3.5 Account Lockout Mechanism 21
2.3.6 Password Reset 21
2.4 Home Page 21
2.4.1 Before Login (Public View) 21
2.4.2 After Login (Authenticated View) 22
2.5 Data Explorer 23
2.5.1 Data Upload Process 23
2.5.2 Supported File Formats 23
2.5.3 Correlation Heatmap 23
2.5.4 Histogram Analysis 24
2.5.5 Statistical Summary 25
2.5.6 Additional Explorer Features 25
2.6 Predict Future Quality 25
2.6.1 Upload Schema Requirements 26
2.6.2 Schema Validation 26
2.6.3 Model Selection 27

3 Developer Documentation 28
3.1 Architecture Overview 28
3.1.1 Layered Architecture 28
3.1.2 Component Interaction Diagram 29
3.1.3 Data Flow 30
3.1.4 Module Organization 30
3.2 Technology Stack 32
3.2.1 Python Ecosystem 32

3.2.2 Streamlit Framework Justification 32
3.2.3 Key Libraries 33
3.3 Data Processing Pipeline 33
3.3.1 Data Loading and Schema Validation 34
3.3.2 Temporal Indexing 34
3.3.3 Missing Value Handling 34
3.3.4 Feature Engineering 35
3.3.5 Normalization 35
3.3.6 Train/Test Splitting Strategy 36
3.4 Model Implementations 36
3.4.1 ARIMA – AutoRegressive Integrated Moving Average 36
3.4.2 LSTM – Long Short-Term Memory Network 38

4 Conclusion **40**
4.1 Summary of Contributions 40
4.2 Model Comparison and Results 42
4.2.1 Quantitative Comparison 42
4.2.2 Model-Specific Observations 42
4.2.3 Key Insights 43
4.3 Limitations 44
4.4 Future Work 46
4.4.1 Ensemble Methods and Model Combination 46
4.4.2 Uncertainty Quantification 46
4.4.3 Transfer Learning and Multi-City Generalization 47
4.4.4 Real-Time Data Ingestion and Operational Deployment 47

Acknowledgements **48**

Bibliography **48**

List of Figures **49**

List of Tables **50**

Chapter 1

Introduction

1.1 Motivation

Air pollution represents one of the most pressing environmental and public health challenges of the twenty-first century. The World Health Organization (WHO) estimates that ambient air pollution contributes to approximately 4.2 million premature deaths annually worldwide, making it the single largest environmental risk factor for human health [**who2021airquality**]. Among the various pollutants that degrade urban air quality, particulate matter with an aerodynamic diameter of less than 2.5 micrometres ($PM_{2.5}$) poses the greatest threat to human well-being due to its ability to penetrate deep into the respiratory system and enter the bloodstream [**pope2006health**].

The health consequences of prolonged exposure to elevated $PM_{2.5}$ concentrations are both severe and wide-ranging. Epidemiological studies have established robust causal links between $PM_{2.5}$ exposure and cardiovascular disease, chronic obstructive pulmonary disease (COPD), lung cancer, stroke, and acute lower respiratory infections [**burnett2014integrated**]. Short-term exposure to pollution spikes has been associated with increased hospital admissions for respiratory and cardiac events, exacerbation of asthma symptoms, and elevated mortality rates among vulnerable populations including the elderly, children, and individuals with pre-existing conditions [**dominici2006fine**]. The WHO's 2021 updated Global Air Quality Guidelines recommend an annual mean $PM_{2.5}$ concentration of no more than 5 $\mu g/m^3$ and a 24-hour mean of no more than 15 $\mu g/m^3$ [**who2021airquality**], thresholds that are routinely exceeded in major urban centres across the globe.

London, as one of Europe's largest and most densely populated metropolitan areas, faces particularly acute air quality challenges. Despite significant improvements since the era of the Great Smog of 1952, the city continues to exceed both WHO guideline values and legal limits set by the UK government and the European Union. Road transport, domestic heating, construction activities, and industrial emissions collectively contribute to a complex pollution landscape that varies substantially across spatial and temporal scales [**beevers2013trends**]. The introduction of the Ultra Low Emission Zone (ULEZ) in 2019 and its subsequent expansions reflect the severity of the problem and the political will to address it, yet monitoring data consistently shows that large portions of the city's population remain exposed to harmful pollution levels [**walton2015understanding**].

The economic burden of air pollution in London is substantial. A report commissioned by the Greater London Authority estimated that the health costs attributable to $PM_{2.5}$ and nitrogen dioxide (NO_2) exposure in London amount to approximately £3.7 billion annually when accounting for healthcare expenditure, lost productivity, and reduced quality of life [**phe2018estimation**]. At the national level, the Royal College of Physicians estimated the annual cost of air pollution to the UK economy at £20 billion [**rcp2016every**]. These figures underscore the economic imperative for effective air quality management strategies, of which accurate forecasting is a critical component.

Accurate prediction of air quality levels enables a range of protective interventions. At the individual level, forecasts allow vulnerable populations to modify their behaviour—remaining indoors during pollution episodes, adjusting medication, or choosing less polluted routes for commuting. At the policy level, reliable predictions support the implementation of emergency measures such as traffic restrictions, industrial curtailment, and public health advisories. Furthermore, forecasting systems contribute to the evaluation of long-term policy interventions by enabling counterfactual analysis and scenario modelling [**zhang2012real**].

The advent of machine learning and deep learning techniques has opened new avenues for air quality prediction that complement and often surpass traditional deterministic and statistical approaches. However, the proliferation of methods raises important questions about model selection, interpretability, and practical deployment. There exists a need for comprehensive platforms that not only generate accurate forecasts but also present them in an accessible manner to diverse stakeholders,

from public health officials to individual citizens. This thesis addresses that need by developing an urban air quality prediction platform for London, leveraging state-of-the-art machine learning methods and providing an interactive interface for forecast exploration.

1.2 Background and Related Work

1.2.1 Time Series Forecasting: Classical Approaches

The prediction of air pollutant concentrations is fundamentally a time series forecasting problem. Classical statistical methods for time series analysis have their roots in the seminal work of Box and Jenkins [**box1970timeseries**], who formalised the Autoregressive Integrated Moving Average (ARIMA) framework. The ARIMA(p, d, q) model captures temporal dependencies through autoregressive terms (AR), differencing for stationarity (I), and moving average components (MA). Extensions of this framework, including Seasonal ARIMA (SARIMA) and ARIMA with exogenous variables (ARIMAX), have been widely applied to environmental time series where periodic patterns and external forcing factors are present [**hyndman2008forecasting**].

The appeal of ARIMA-family models lies in their well-understood theoretical properties, interpretability, and relatively modest data requirements. Hyndman and Khandakar [**hyndman2008forecasting**] developed automated procedures for ARIMA model selection based on information criteria, making these methods accessible to practitioners without deep expertise in time series econometrics. In the air quality domain, numerous studies have employed ARIMA models for short-term pollutant forecasting. Kumar and Jain [**kumar2010arima**] applied SARIMA to predict daily PM_{10} concentrations in Delhi, achieving reasonable accuracy for one-day-ahead forecasts. Similarly, Díaz-Robles et al. [**diaz2008comparison**] compared ARIMA with neural network approaches for $PM_{2.5}$ prediction in Temuco, Chile.

However, classical time series methods face well-documented limitations when applied to air quality data. The assumption of linearity is frequently violated, as pollutant concentrations exhibit complex nonlinear relationships with meteorological variables and emission sources. Furthermore, ARIMA models struggle to capture long-range dependencies and abrupt regime changes that characterise pollution

episodes [**freeman2018forecasting**]. The stationarity assumption, while addressable through differencing, may not adequately represent the non-stationary dynamics of urban air quality systems influenced by evolving emission patterns and climate variability.

1.2.2 Deep Learning Approaches

The limitations of linear statistical models have motivated the application of deep learning architectures to air quality forecasting. Recurrent Neural Networks (RNNs), designed to process sequential data by maintaining hidden states that encode temporal context, represent a natural choice for time series prediction tasks. However, standard RNNs suffer from the vanishing gradient problem, which limits their ability to learn long-range temporal dependencies [**bengio1994learning**].

The Long Short-Term Memory (LSTM) architecture, introduced by Hochreiter and Schmidhuber [**hochreiter1997lstm**], addresses this limitation through a gating mechanism that selectively retains or discards information across time steps. The LSTM cell comprises input, forget, and output gates that regulate the flow of information, enabling the network to maintain relevant context over extended sequences. This capability is particularly valuable for air quality prediction, where pollutant concentrations may be influenced by meteorological conditions and emission patterns from hours or days prior.

Numerous studies have demonstrated the effectiveness of LSTM networks for air quality forecasting. Zheng et al. [**zheng2015forecasting**] developed a deep learning framework for real-time air quality prediction in Chinese cities, incorporating spatial and temporal features through a combination of neural network architectures. Freeman et al. [**freeman2018forecasting**] applied LSTM networks to forecast $PM_{2.5}$ concentrations in London, reporting improvements over persistence and ARIMA baselines. Li et al. [**li2017long**] employed a stacked LSTM architecture for multi-step-ahead prediction of $PM_{2.5}$ in Beijing, demonstrating the model's ability to capture complex temporal patterns during severe pollution episodes.

Variants and extensions of the LSTM architecture have also been explored. Bidirectional LSTMs process sequences in both forward and backward directions, potentially capturing dependencies that unidirectional models miss. Encoder-decoder architectures with attention mechanisms have been applied to multi-step fore-

casting, allowing the model to selectively attend to relevant historical time steps when generating predictions [**qin2017dual**]. More recently, Transformer-based architectures [**vaswani2017attention**], originally developed for natural language processing, have been adapted for time series forecasting with promising results, though their application to air quality prediction remains an active area of research [**liang2023airformer**].

Despite their predictive power, deep learning models present challenges in terms of interpretability, computational requirements, and the need for large training datasets. The "black box" nature of neural networks can limit their acceptance by domain experts and policymakers who require understanding of the factors driving predictions. These considerations motivate the exploration of alternative machine learning approaches that may offer competitive accuracy with greater transparency.

1.2.3 Tree-Based Ensemble Methods

Gradient-boosted decision tree ensembles have emerged as powerful alternatives to deep learning for structured tabular data, including environmental time series. XGBoost (eXtreme Gradient Boosting), introduced by Chen and Guestrin [**chen2016xgboost**], implements a scalable and regularised gradient boosting framework that has achieved state-of-the-art results across numerous machine learning competitions and applied domains. The algorithm constructs an ensemble of decision trees sequentially, with each tree trained to correct the residual errors of the preceding ensemble, while incorporating L1 and L2 regularisation to prevent overfitting.

Random Forest [**breiman2001random**], another prominent ensemble method, constructs multiple decision trees through bootstrap aggregation (bagging) and random feature subsampling. While typically less prone to overfitting than individual decision trees, Random Forests may require larger ensembles to achieve the same predictive accuracy as gradient-boosted methods on complex tasks.

The application of tree-based methods to air quality prediction has yielded competitive results. Castelli et al. [**castelli2020machine**] compared XGBoost, Random Forest, and neural network approaches for $PM_{2.5}$ prediction, finding that XGBoost achieved the best performance in terms of root mean squared error (RMSE) while requiring substantially less training time than deep learning alternatives. Gu et al.

[**gu2020stacking**] proposed a stacking ensemble combining XGBoost with other base learners for air quality index prediction, demonstrating that ensemble diversity improves generalisation.

A significant advantage of tree-based methods is their native support for feature importance analysis. SHAP (SHapley Additive exPlanations) values [**lundberg2017unified**] provide theoretically grounded attributions of individual predictions to input features, enabling practitioners to understand which meteorological variables, temporal features, or lagged pollutant values drive specific forecasts. This interpretability is valuable for validating model behaviour against domain knowledge and for communicating results to non-technical stakeholders.

1.2.4 Existing Air Quality Platforms and Applications

Several operational air quality forecasting systems and public-facing platforms exist at various scales. The European Copernicus Atmosphere Monitoring Service (CAMS) provides continental-scale air quality forecasts based on chemical transport models, offering predictions up to five days ahead for major pollutants [**marecal2015regional**]. In the United Kingdom, the Department for Environment, Food and Rural Affairs (DEFRA) operates the Daily Air Quality Index (DAQI) system, which provides forecasts and health advice based on a combination of numerical weather prediction and statistical post-processing.

At the urban scale, several cities have developed bespoke air quality information systems. The London Air Quality Network (LAQN) provides real-time monitoring data and basic forecasts through its web interface [**laqn2024**]. Commercial applications such as BreezoMeter, Plume Labs, and IQAir offer hyperlocal air quality information by combining monitoring data with proprietary models. However, these platforms typically do not disclose their methodological details, limiting reproducibility and scientific scrutiny.

Academic contributions to air quality platform development include the work of Zheng et al. [**zheng2015forecasting**], who developed U-Air, a system for fine-grained urban air quality inference and prediction in Chinese cities. Their approach combined data from sparse monitoring stations with urban computing techniques to estimate pollution levels at unmonitored locations. Yi et al. [**yi2018deep**] extended this work with deep learning architectures capable of capturing spatio-temporal

correlations across monitoring networks.

1.2.5 Positioning of This Work

The present thesis distinguishes itself from existing work in several respects. First, it provides a systematic comparison of classical statistical methods (ARIMA), deep learning approaches (LSTM), and tree-based ensemble methods (XGBoost) within a unified experimental framework applied to London air quality data. While individual studies have applied these methods in isolation, comprehensive comparisons using consistent data preprocessing, evaluation metrics, and temporal splits are less common. Second, this work emphasises the development of a complete prediction platform rather than focusing solely on model accuracy. The platform integrates data ingestion, preprocessing, model training, prediction generation, and interactive visualisation into a cohesive system suitable for practical deployment. Third, the thesis addresses the specific characteristics of London's air quality monitoring infrastructure, including data quality challenges, network coverage, and the influence of local meteorological conditions on pollutant dispersion.

1.3 Problem Statement

Urban air quality prediction can be formally defined as follows. Let $\mathbf{x}_t = (x_t^{(1)}, x_t^{(2)}, \ldots, x_t^{(n)})$ denote the vector of observed pollutant concentrations and meteorological variables at time step t, where n is the number of measured variables. Given a historical observation window $\mathbf{X} = \{\mathbf{x}_{t-w+1}, \mathbf{x}_{t-w+2}, \ldots, \mathbf{x}_t\}$ of length w, the forecasting task is to predict the target pollutant concentration y_{t+h} at a future horizon h:

$$\hat{y}_{t+h} = f(\mathbf{X}; \boldsymbol{\theta}) \tag{1.1}$$

where f is the learned forecasting function parameterised by $\boldsymbol{\theta}$, and $\hat{y}_{t+h}$ is the predicted $PM_{2.5}$ concentration at time $t + h$. The objective is to minimise a loss function $\mathcal{L}(y_{t+h}, \hat{y}_{t+h})$ over a held-out test set, where common choices include mean squared error (MSE) and mean absolute error (MAE).

Despite the extensive literature on air quality forecasting, several gaps persist that this thesis aims to address:

1. **Lack of unified comparison frameworks.** Studies typically evaluate a single method or a narrow subset of approaches, making it difficult to draw reliable conclusions about relative performance. Differences in data sources, preprocessing pipelines, evaluation periods, and metrics further complicate cross-study comparisons.

2. **Limited focus on deployment.** The majority of academic work prioritises predictive accuracy without addressing the engineering challenges of deploying models as part of an operational system. Issues such as data pipeline reliability, model retraining schedules, and user interface design receive insufficient attention.

3. **Insufficient attention to London-specific challenges.** While London benefits from a dense monitoring network, the data exhibit characteristics—including frequent sensor failures, calibration drift, and complex spatial heterogeneity—that require tailored preprocessing and modelling strategies.

4. **Accessibility gap.** Existing platforms either lack methodological transparency (commercial applications) or lack user-friendly interfaces (academic prototypes). There is a need for systems that combine rigorous methodology with accessible presentation.

This thesis addresses these gaps by developing a comprehensive urban air quality prediction platform that systematically compares multiple forecasting methodologies, implements robust data processing pipelines tailored to London's monitoring infrastructure, and provides an interactive web-based interface for forecast exploration and model comparison.

1.4 Objectives

The specific objectives of this thesis are as follows:

1. To design and implement a data ingestion and preprocessing pipeline for London Air Quality Network data, incorporating automated quality control, missing value imputation, and feature engineering.

2. To implement and evaluate three distinct forecasting approaches—ARIMA, LSTM, and XGBoost—for multi-horizon $PM_{2.5}$ prediction at London monitoring stations.

3. To conduct a rigorous comparative analysis of model performance using appropriate evaluation metrics, statistical significance tests, and analysis of failure modes.

4. To develop an interactive web-based platform that integrates the trained models and provides real-time and forecast air quality information to end users.

5. To analyse feature importance and model interpretability, identifying the key drivers of $PM_{2.5}$ variability in London and assessing the physical plausibility of learned relationships.

6. To evaluate the complete system in terms of prediction accuracy, computational efficiency, and usability, identifying limitations and directions for future improvement.

1.5 Dataset Description

1.5.1 The London Air Quality Network

The primary data source for this thesis is the London Air Quality Network (LAQN), a comprehensive monitoring infrastructure operated by the Environmental Research Group at Imperial College London [**laqn2024**]. Established in 1993, the LAQN comprises over 100 active monitoring sites distributed across Greater London's 33 boroughs, making it one of the densest urban air quality monitoring networks in Europe. The network serves multiple purposes: regulatory compliance assessment, public information provision, epidemiological research support, and policy evaluation.

Monitoring sites within the LAQN are classified according to their proximity to emission sources and the population they represent. *Background* sites are located away from direct emission sources and represent the general urban pollution level experienced by the majority of the population. *Roadside* sites are positioned within 1–5 metres of busy roads and capture the elevated concentrations associated with

traffic emissions. *Kerbside* sites are located at the edge of the road and measure the highest concentrations to which pedestrians and cyclists may be exposed. This classification is important for model development, as the temporal dynamics and predictor relationships differ substantially between site types.

1.5.2 Measured Variables

The LAQN monitors a comprehensive suite of pollutants and meteorological parameters. The pollutant measurements relevant to this thesis include:

- **$PM_{2.5}$** — Particulate matter with aerodynamic diameter < 2.5 μm, measured in $\mu g/m^3$. This is the primary prediction target. Measurements are obtained using Tapered Element Oscillating Microbalance (TEOM) or Filter Dynamics Measurement System (FDMS) instruments, with Beta Attenuation Monitors (BAM) deployed at some sites.
- **PM_{10}** — Particulate matter with aerodynamic diameter < 10 μm, measured in $\mu g/m^3$. Serves as a predictor variable due to its correlation with $PM_{2.5}$.
- **NO_2** — Nitrogen dioxide, measured in $\mu g/m^3$. A key traffic-related pollutant that shares emission sources with $PM_{2.5}$.
- **O_3** — Ozone, measured in $\mu g/m^3$. Exhibits an inverse relationship with NO_x in urban environments due to titration chemistry.

Meteorological variables, obtained from co-located weather stations or nearby Met Office observations, include temperature, relative humidity, wind speed, wind direction, atmospheric pressure, and precipitation. These variables are critical predictors of pollutant concentrations, as they govern atmospheric dispersion, chemical transformation, and deposition processes.

1.5.3 Temporal Coverage and Resolution

The dataset used in this thesis spans the period from January 2015 to December 2023, providing nine years of observations that capture both long-term trends and inter-annual variability. Data are available at hourly temporal resolution, yielding approximately 78,840 potential observations per variable per site over the study period. For modelling purposes, both hourly and daily aggregated time series are

considered, with the choice of temporal resolution depending on the forecasting horizon and application requirements.

The study period encompasses several events of relevance to air quality modelling, including the introduction and expansion of the ULEZ (April 2019, October 2021), the COVID-19 pandemic lockdowns (March–June 2020, January–March 2021), and various meteorological extremes including heatwaves and Saharan dust episodes. These events introduce non-stationarity and distributional shifts that challenge forecasting models and provide opportunities to assess model robustness.

1.5.4 Data Quality Considerations

Raw monitoring data from the LAQN exhibit several quality issues that necessitate careful preprocessing:

- **Missing data.** Instrument failures, calibration periods, and communication outages result in gaps ranging from isolated hourly values to extended periods of weeks or months. The data capture rate varies by site and pollutant, with typical annual capture rates ranging from 75% to 95% for well-maintained sites.

- **Calibration artefacts.** Periodic instrument calibration can introduce step changes or brief periods of anomalous readings. Automatic and manual calibration events are flagged in the raw data but require careful handling during preprocessing.

- **Negative values.** Some measurement techniques, particularly TEOM instruments, can produce negative concentration values under certain conditions (e.g., volatile particle loss). These physically implausible values require identification and treatment.

- **Outliers and spikes.** Transient instrument malfunctions, local emission events (e.g., nearby construction or fires), and data transmission errors can produce extreme values that do not represent ambient conditions.

- **Measurement method changes.** Over the nine-year study period, some sites have undergone instrument upgrades or changes in measurement methodology, potentially introducing inhomogeneities in the time series.

The data preprocessing pipeline developed in this thesis (described in Chapter **??**) addresses these issues through a combination of automated quality flags, statistical outlier detection, interpolation for short gaps, and exclusion criteria for extended missing periods. The impact of preprocessing choices on model performance is assessed through sensitivity analysis.

1.6 Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter ??** presents the methodology, including detailed descriptions of the data preprocessing pipeline, feature engineering procedures, and the three forecasting models (ARIMA, LSTM, XGBoost). Hyperparameter tuning strategies and evaluation metrics are also discussed.
- **Chapter ??** describes the system architecture and implementation of the prediction platform, including the technology stack, data flow, model serving infrastructure, and web interface design.
- **Chapter ??** presents the experimental results, including model comparison across multiple forecasting horizons, feature importance analysis, and case studies of prediction performance during notable air quality events.
- **Chapter ??** summarises the findings, discusses limitations, and outlines directions for future work including spatial modelling extensions, additional pollutant targets, and real-time deployment considerations.

Chapter 2

User Documentation

This chapter provides comprehensive documentation for end users of the Urban Air Quality Forecasting application. The application is built as a multi-page Streamlit web application that enables users to explore air quality data, generate forecasts using machine learning models, and monitor urban air pollution trends through interactive dashboards.

2.1 System Requirements

2.1.1 Hardware Requirements

Table 2.1: Minimum and recommended hardware specifications

Component	**Minimum**	**Recommended**
Processor	Dual-core 2.0 GHz	Quad-core 3.0 GHz+
RAM	4 GB	8 GB+
Storage	2 GB free space	5 GB+ free space
Network	Broadband connection	Stable broadband (10 Mbps+)
GPU	Not required	CUDA-compatible (for training)

2.1.2 Software Requirements

- **Operating System:** Windows 10/11, macOS 12+, or Ubuntu 20.04+
- **Python:** Version 3.9 or higher (3.11 recommended)
- **pip:** Version 21.0 or higher

- **Git:** Version 2.30+ (for cloning the repository)
- **Database:** SQLite 3.35+ (bundled with Python) or PostgreSQL 13+ (optional, for production)

2.1.3 Browser Requirements

The application is tested and supported on the following browsers:

- Google Chrome 90+ (recommended)
- Mozilla Firefox 88+
- Microsoft Edge 90+
- Safari 14+

JavaScript must be enabled, and a minimum viewport width of 1024 pixels is recommended for optimal dashboard rendering. The application is responsive but designed primarily for desktop use.

2.2 Installation and Configuration

2.2.1 Cloning the Repository

Begin by cloning the project repository to your local machine:

2.2.2 Setting Up the Virtual Environment

It is strongly recommended to use a Python virtual environment to isolate project dependencies from the system Python installation.

On Linux/macOS

On Windows

2.2.3 Installing Dependencies

The project uses a `requirements.txt` file to manage all Python dependencies: The key dependencies include:

Table 2.2: Core project dependencies

Package	Version	Purpose
streamlit	$\geq$ 1.28.0	Web application framework
pandas	$\geq$ 2.0.0	Data manipulation
scikit-learn	$\geq$ 1.3.0	Machine learning models
xgboost	$\geq$ 2.0.0	Gradient boosting models
plotly	$\geq$ 5.17.0	Interactive visualizations
google-generativeai	$\geq$ 0.3.0	Gemini API integration
bcrypt	$\geq$ 4.0.0	Password hashing
streamlit-option-menu	$\geq$ 0.3.6	Navigation components

2.2.4 Configuring Secrets

Streamlit uses a secrets management system via the `.streamlit/secrets.toml` file. Create this file in the project root:

Populate the secrets file with the required configuration:

Important: Never commit the `secrets.toml` file to version control. Ensure it is listed in your `.gitignore` file.

2.2.5 Database Initialization

The application automatically initializes the SQLite database on first run. However, you can manually trigger initialization:

This creates the necessary tables for user accounts, prediction history, and application settings.

2.2.6 Model Training

Before generating predictions, the machine learning models must be trained on historical air quality data. The training pipeline supports multiple model architectures:

Upon successful training, model artifacts are saved to the `models/` directory as serialized `.pkl` files. Training metrics (RMSE, MAE, R^2) are logged to the console and saved to `models/training_report.json`.

2.2.7 Running the Application

Launch the application using the Streamlit CLI:

```
streamlit run front_end/Home.py
```

The application will be accessible at `http://localhost:8501`. On first launch, Streamlit may ask permission to collect usage statistics—this is optional and can be declined.

2.2.8 Production Deployment Considerations

For production environments, additional configuration is recommended:

2.3 Authentication System

The application implements a comprehensive authentication system to protect user data and personalize the forecasting experience. All passwords are hashed using the bcrypt algorithm before storage, ensuring that plaintext credentials are never persisted.

2.3.1 Registration Flow

New users can create an account through the registration form accessible from the login page. The registration process follows these steps:

1. Navigate to the application URL and click **"Create Account"** on the login page.
2. Fill in the required fields:
   - **Username:** 3–20 characters, alphanumeric and underscores only
   - **Email:** Valid email format (used for account recovery)
   - **Full Name:** Display name shown in the application
   - **Password:** Must meet complexity requirements (see below)
   - **Confirm Password:** Must match the password field exactly
3. Click **"Register"** to submit the form.
4. Upon successful registration, the user is redirected to the login page with a confirmation message.

2.3.2 Password Requirements

The application enforces the following password policy:

Table 2.3: Password complexity requirements

Requirement	Details
Minimum length	8 characters
Maximum length	128 characters
Uppercase letters	At least 1 required
Lowercase letters	At least 1 required
Numeric digits	At least 1 required
Special characters	At least 1 required (!@#$%^&*)
Common passwords	Rejected (checked against blocklist)

A real-time password strength indicator provides visual feedback during registration, displaying strength levels from "Weak" (red) through "Fair" (orange) to "Strong" (green).

2.3.3 Login Process

Returning users authenticate through the login form:

1. Enter the registered username or email address.
2. Enter the account password.
3. Optionally check **"Remember me"** to extend session duration.
4. Click **"Login"** to authenticate.

Upon successful authentication, the user is redirected to the Home page and the navigation sidebar becomes fully accessible.

2.3.4 Session Persistence

The application manages user sessions through Streamlit's session state mechanism:

- **Default session:** Expires after 30 minutes of inactivity.
- **"Remember me" session:** Persists for 7 days via a browser cookie.

- **Manual logout:** Users can explicitly end their session via the sidebar logout button.
- **Session tokens:** Cryptographically generated tokens are stored server-side and validated on each request.

2.3.5 Account Lockout Mechanism

To prevent brute-force attacks, the application implements progressive account lockout:

1. After **3 failed attempts**: A CAPTCHA challenge is displayed.
2. After **5 failed attempts**: The account is locked for 15 minutes.
3. After **10 failed attempts**: The account is locked for 60 minutes and an email notification is sent (if configured).

The lockout counter resets after a successful login. Administrators can manually unlock accounts through the database if needed:

2.3.6 Password Reset

Users who forget their password can initiate a reset through the **"Forgot Password?"** link on the login page. The reset flow generates a time-limited token (valid for 1 hour) and displays it to the user for demonstration purposes. In a production environment, this token would be sent via email.

2.4 Home Page

The Home page serves as the application's landing page and adapts its content based on the user's authentication status.

2.4.1 Before Login (Public View)

Unauthenticated visitors see:

- **Application title and tagline:** "Urban Air Quality Forecasting — Monitor, Analyze, and Predict Air Pollution"

- **Feature highlights:** A grid of cards summarizing key capabilities (data exploration, ML predictions, real-time dashboard, history tracking)
- **Quick statistics:** Aggregate metrics such as total predictions made, active users, and supported pollutants
- **Call-to-action buttons:** Prominent "Login" and "Register" buttons directing users to the authentication page
- **Technology stack:** Brief overview of the technologies powering the application

2.4.2 After Login (Authenticated View)

Authenticated users see a personalized dashboard summary:

- **Welcome message:** Personalized greeting with the user's display name and last login timestamp
- **Activity summary cards:** Number of predictions made, datasets uploaded, and days since registration
- **Recent predictions:** A compact table showing the last 5 predictions with timestamps and model types
- **Quick navigation:** Large icon buttons providing one-click access to each application module
- **Air quality tip of the day:** Rotating educational content about air pollution and health impacts
- **System announcements:** Any active notifications from administrators (e.g., scheduled maintenance)

The Home page layout uses Streamlit's column system to create a responsive grid that adapts to different screen sizes. Key metrics are displayed using `st.metric()` components with delta indicators showing changes from the previous period.

2.5 Data Explorer

The Data Explorer module provides tools for uploading, visualizing, and statistically analyzing air quality datasets. It is designed to help users understand their data before proceeding to the prediction module.

2.5.1 Data Upload Process

Users can upload datasets through the file uploader widget:

1. Navigate to the **"Data Explorer"** page from the sidebar.
2. Click the **"Browse files"** button or drag and drop a file into the upload area.
3. The application validates the file format and displays a preview of the first 10 rows.
4. If validation succeeds, the full dataset is loaded into memory for analysis.

2.5.2 Supported File Formats

Table 2.4: Supported data file formats

Format	Extension	Max Size	Notes
CSV	.csv	50 MB	Comma, semicolon, or tab delimited
Excel	.xlsx, .xls	50 MB	First sheet read by default
JSON	.json	50 MB	Records or columnar orientation
Parquet	.parquet	100 MB	Recommended for large datasets

The application automatically detects the delimiter for CSV files and handles common encoding issues (UTF-8, Latin-1). Column names are normalized by stripping whitespace and converting to lowercase for consistency.

2.5.3 Correlation Heatmap

The correlation heatmap visualizes pairwise Pearson correlation coefficients between all numeric columns in the dataset:

- **Color scale:** Diverging color map from blue (-1) through white (0) to red ($+1$).

- **Annotations:** Each cell displays the correlation coefficient rounded to two decimal places.
- **Interactivity:** Hovering over a cell shows the exact value and the two correlated variables.
- **Filtering:** Users can select a subset of columns to include in the heatmap via a multiselect widget.
- **Threshold highlighting:** An optional slider allows users to highlight only correlations above a specified absolute threshold.

The heatmap is rendered using Plotly, enabling zoom, pan, and export to PNG. This visualization is particularly useful for identifying multicollinearity between pollutant measurements and meteorological variables.

2.5.4 Histogram Analysis

The histogram module allows users to explore the distribution of individual variables:

- **Column selection:** A dropdown menu lists all numeric columns available for plotting.
- **Bin count:** Adjustable slider (10–100 bins) to control histogram granularity.
- **Distribution overlay:** Optional kernel density estimation (KDE) curve overlaid on the histogram.
- **Normalization:** Toggle between raw counts and probability density.
- **Group by:** Optional categorical column for creating grouped/stacked histograms.
- **Log scale:** Toggle for logarithmic y-axis, useful for skewed distributions.

Users can generate histograms for multiple columns simultaneously using the "Plot All" button, which creates a grid of subplots for quick visual comparison.

Table 2.5: Statistical metrics computed for each numeric column

Metric	Description
Count	Number of non-null values
Mean	Arithmetic mean
Std	Standard deviation
Min / Max	Minimum and maximum values
25% / 50% / 75%	Quartile values
Skewness	Measure of distribution asymmetry
Kurtosis	Measure of distribution tail heaviness
Missing (%)	Percentage of null/NaN values

2.5.5 Statistical Summary

The statistical summary panel provides comprehensive descriptive statistics: The summary is displayed as an interactive dataframe with sortable columns. Users can download the statistical summary as a CSV file using the **"Download Summary"** button.

2.5.6 Additional Explorer Features

- **Data type detection:** Automatic identification of datetime, numeric, and categorical columns with suggested type conversions.
- **Missing value visualization:** A bar chart showing the percentage of missing values per column, with options to view the missingness pattern matrix.
- **Scatter plot matrix:** Pairwise scatter plots for selected variables with optional color coding by a categorical variable.
- **Time series plot:** If a datetime column is detected, users can plot any numeric variable over time with resampling options (hourly, daily, weekly, monthly).
- **Outlier detection:** IQR-based outlier flagging with visual indicators on box plots.

2.6 Predict Future Quality

The Predict Future Quality module is the core forecasting engine of the application. It allows users to upload new air quality measurements and generate predictions using pre-trained machine learning models.

2.6.1 Upload Schema Requirements

The prediction module requires input data to conform to a specific schema. The uploaded CSV file must contain the following columns:

Table 2.6: Required columns for prediction input

Column Name	**Type**	**Unit**	**Required**
datetime	datetime	ISO 8601	Yes
pm25	float	$\mu g/m^3$	Yes
pm10	float	$\mu g/m^3$	Yes
no2	float	ppb	Yes
so2	float	ppb	Conditional
co	float	ppm	Conditional
o3	float	ppb	Conditional
temperature	float	°C	Recommended
humidity	float	%	Recommended
wind_speed	float	m/s	Recommended

Columns marked "Conditional" are required depending on the selected prediction target. Columns marked "Recommended" improve prediction accuracy but are not strictly required—the model will use default imputation strategies for missing meteorological features.

2.6.2 Schema Validation

Upon upload, the application performs comprehensive validation:

1. **Column presence check:** Verifies all required columns exist (case-insensitive matching).
2. **Data type validation:** Ensures numeric columns contain parseable values.
3. **Range validation:** Flags physically impossible values (e.g., negative concentrations, humidity $> 100\%$).
4. **Temporal ordering:** Checks that datetime values are monotonically increasing.
5. **Minimum row count:** Requires at least 24 rows (representing 24 hours of data) for meaningful predictions.

Validation errors are displayed as an itemized list with specific row/column references, enabling users to quickly identify and correct issues in their data.

2.6.3 Model Selection

Chapter 3

Developer Documentation

This chapter provides a comprehensive technical description of the air quality forecasting platform's implementation. It covers the system architecture, technology choices, data processing pipelines, model implementations, database design, security mechanisms, and the web application layer.

3.1 Architecture Overview

3.1.1 Layered Architecture

The application follows a layered architecture pattern that separates concerns into distinct tiers, each responsible for a well-defined subset of functionality. This separation enables independent development, testing, and maintenance of each layer while maintaining clear interfaces between them.

The system is organized into four primary layers:

1. **Presentation Layer** – The Streamlit-based web interface responsible for rendering interactive visualizations, handling user input, and managing page navigation. This layer communicates exclusively with the business logic layer and never directly accesses the database or model artifacts.

2. **Business Logic Layer** – Contains the core application logic including model training orchestration, forecast generation, user management, and data validation. This layer acts as a mediator between the presentation and data layers.

3. **Data Access Layer** – Provides an abstraction over the SQLite database through dedicated repository classes. All SQL queries are encapsulated within

this layer, exposing only high-level CRUD operations to the business logic layer.

4. **Infrastructure Layer** – Encompasses external dependencies including the file system (for model serialization), the SQLite database engine, and third-party APIs for data acquisition.

3.1.2 Component Interaction Diagram

The following describes the component interaction topology of the system:

```
+-------------------------------------------------------------+
|              front_end/ (Presentation Layer)                 |
|  +------------+  +-------------+  +-------------------+  |
|  | Dashboard  |  | Predict     |  | Data Explorer     |  |
|  | History    |  | HelpDesk    |  | Settings          |  |
|  +------+-----+  +------+------+  +---------+---------+  |
+---------|--------------|-----------------------|----------+
          |              |                       |
+---------v--------------v-----------------------v----------+
|              back_end/ (Business Logic Layer)                |
|  +----------------+  +---------------+  +-------------+  |
|  | forecasting.py |  | data_proc.py  |  | helpdesk    |  |
|  | (LSTM/XGBoost) |  | (validation)  |  | (Gemini)    |  |
|  +-------+--------+  +-------+-------+  +------+------+  |
+---------|-----------------------|----------------|---------+
          |                       |                |
+---------v-----------------------v----------------v---------+
|              db/ (Data Access Layer)                          |
|  +-------------------------------------------------------+  |
|  | database.py: users, settings, history, chat, tokens |  |
|  +-------------------------------------------------------+  |
+-------------------------------------------------------------+
          |                       |
+---------v-----------------------v-------------------------+
|              Infrastructure                                  |
```

```
|  +-------------+  +----------------+  +---------------+   |
|  | SQLite DB   |  | models/        |  | Gemini API    |   |
|  | (db/)       |  | (.keras,       |  | (external)    |   |
|  |             |  |  .joblib)      |  |               |   |
|  +-------------+  +----------------+  +---------------+   |
+-------------------------------------------------------------+
```

3.1.3 Data Flow

The primary data flow through the system follows a pipeline pattern:

1. **Data Ingestion**: Raw CSV or API data enters the system through the presentation layer's file upload mechanism or scheduled data fetching routines.

2. **Validation and Transformation**: The business logic layer validates the schema, handles missing values, performs temporal indexing, and engineers features appropriate for each model type.

3. **Model Training**: Processed data flows into the model training pipeline where it is split into training and testing sets, normalized as required, and used to fit the selected forecasting model.

4. **Forecast Generation**: Trained models generate predictions that flow back through the business logic layer, are stored in the database, and are rendered as interactive visualizations in the presentation layer.

5. **Persistence**: Model artifacts, training metadata, and forecast results are persisted to the file system and database respectively, enabling reproducibility and historical comparison.

The system employs an event-driven interaction model within the Streamlit framework, where user actions (button clicks, parameter changes) trigger re-execution of the relevant computation pipeline. Session state is used to cache intermediate results and prevent unnecessary recomputation.

3.1.4 Module Organization

The project follows a layered architecture with clear separation between frontend, backend, database, and model concerns:

```
project_root/
--- front_end/              # Presentation layer (Streamlit)
-   --- Home.py             # Entry point
-   --- ui.py               # Shared layout, theme, session mgmt
-   --- charts.py           # Plotly configuration
-   --- pages/
-       --- 1_Data_Explorer.py
-       --- 2_Predict_future_quality.py
-       --- 3_Dashboard.py
-       --- 4_History.py
-       --- 5_Settings.py
-       --- 6_HelpDesk.py
-   --- static/theme.css    # Minimal CSS theme
--- back_end/               # Business logic layer
-   --- forecasting.py      # Model loading & 168h forecast
-   --- data_processing.py  # Data validation & preprocessing
-   --- page_dashboard.py   # Dashboard chart logic
-   --- page_predict.py     # Forecast figure builder
-   --- page_helpdesk.py    # Chat UI logic
-   --- helpdesk_gemini.py  # Gemini API client
--- db/                     # Data access layer
-   --- database.py         # SQLite operations (users, history,
-                           #   settings, chat, auth tokens)
--- models/                 # ML model layer
-   --- lstm_multivariate_model.keras
-   --- lstm_multivariate_artifacts.joblib
-   --- xgboost_multivariate_artifacts.joblib
-   --- lstm_forecast.py    # Standalone LSTM evaluation
-   --- xgboost_forecast.py # Standalone XGBoost evaluation
-   --- notebooks/          # Training scripts
--- data/                   # Datasets & preprocessing
-   --- london_2024.csv     # Primary dataset (hourly)
--- tests/                  # Pytest test suite
```

3.2 Technology Stack

3.2.1 Python Ecosystem

Python 3.10+ was selected as the primary development language for several compelling reasons:

- **Scientific Computing Ecosystem**: Python offers the most mature and comprehensive ecosystem for data science and machine learning, with libraries such as NumPy, pandas, scikit-learn, TensorFlow, and XGBoost providing production-ready implementations of statistical and machine learning algorithms.
- **Rapid Prototyping**: Python's dynamic typing and expressive syntax enable rapid iteration during the research and development phase, which is critical for a thesis project with time constraints.
- **Community and Documentation**: The extensive community support and documentation available for Python's data science stack significantly reduces development friction.
- **Type Hinting Support**: Python 3.10+ provides robust type hinting capabilities that, combined with static analysis tools like mypy, offer many benefits of static typing without sacrificing development speed.

3.2.2 Streamlit Framework Justification

Streamlit was chosen over alternatives such as Flask+React, Dash, or Gradio for the following reasons:

Criterion	**Streamlit**	**Dash**	**Flask+React**	**Gradio**
Development Speed	Excellent	Good	Poor	Excellent
Interactive Widgets	Built-in	Built-in	Manual	Built-in
Data Visualization	Native	Native	Manual	Limited
Customizability	Moderate	High	Full	Low
Python-only Stack	Yes	Yes	No	Yes
Session Management	Built-in	Manual	Manual	Limited
Multi-page Support	Native	Manual	Manual	Limited

Table 3.1: Web framework comparison for data science applications

Streamlit's declarative programming model, where the script re-executes top-to-bottom on each interaction, aligns naturally with the data pipeline paradigm. Its built-in caching mechanism (`@st.cache_data`, `@st.cache_resource`) provides efficient memoization of expensive computations without requiring external caching infrastructure.

3.2.3 Key Libraries

Library	Version	Purpose
pandas	≥2.0	DataFrame operations, temporal indexing, data manipulation
plotly	≥5.15	Interactive time series visualizations with zoom, pan, and hover
tensorflow	≥2.13	LSTM neural network implementation and training
xgboost	≥1.7	Gradient boosted tree ensemble model
statsmodels	≥0.14	ARIMA model fitting and statistical tests
scikit-learn	≥1.3	Preprocessing (MinMaxScaler), metrics, train/test splitting
bcrypt	≥4.0	Secure password hashing with adaptive cost factor
sqlite3	stdlib	Embedded relational database (Python standard library)
streamlit	≥1.28	Web application framework
numpy	≥1.24	Numerical array operations

Table 3.2: Primary library dependencies and their roles

The choice of Plotly over Matplotlib for visualization is motivated by Plotly's native interactivity (zooming into specific time ranges is essential for air quality analysis) and its seamless integration with Streamlit via `st.plotly_chart()`.

3.3 Data Processing Pipeline

The data processing pipeline transforms raw air quality measurements into model-ready feature matrices. This section details each stage of the pipeline, from initial loading through to the final train/test split.

3.3.1 Data Loading and Schema Validation

Raw data enters the system as CSV files containing timestamped air quality measurements. The loading procedure enforces a strict schema validation protocol:

The validation process performs the following checks:

- Presence of all required columns
- Type coercion to expected data types
- Range validation (e.g., PM2.5 values must be non-negative)
- Duplicate timestamp detection and removal
- Chronological ordering verification

3.3.2 Temporal Indexing

After schema validation, the timestamp column is converted to a `DatetimeIndex` and set as the DataFrame index. This enables pandas' powerful time series functionality:

The frequency inference step is critical for ARIMA modeling, which requires regularly-spaced observations. When the inferred frequency is `None` (indicating irregular spacing), the data is resampled to hourly resolution using mean aggregation.

3.3.3 Missing Value Handling

Air quality sensor data frequently contains gaps due to sensor malfunctions, maintenance periods, or communication failures. The pipeline employs a multi-strategy approach to handle missing values:

1. **Short gaps** (≤ 3 consecutive missing values): Linear interpolation is applied, as short gaps in air quality data typically exhibit smooth transitions.
2. **Medium gaps** (4–24 consecutive missing values): Forward fill followed by backward fill is used, preserving the last known state while avoiding extrapolation artifacts.
3. **Long gaps** (> 24 consecutive missing values): These segments are flagged and optionally excluded from training data, as imputation over such long periods would introduce significant bias.

3.3.4 Feature Engineering

Each model type requires different feature representations. The pipeline generates model-specific feature sets:

ARIMA Features

ARIMA operates on univariate time series, so feature engineering is minimal:

- Selection of the target pollutant column
- Stationarity testing via the Augmented Dickey-Fuller (ADF) test
- Differencing order determination based on ADF p-values
- Optional log transformation for variance stabilization

LSTM Features

The LSTM model requires normalized, windowed input sequences:

XGBoost Features

XGBoost requires tabular features with explicit lag and rolling statistics:

3.3.5 Normalization

Normalization strategies differ by model:

Model	Normalization	Rationale
ARIMA	None	ARIMA operates on raw values; stationarity is achieved through differencing
LSTM	Min-Max [0,1]	Neural networks converge faster with bounded inputs; sigmoid/-tanh activations expect normalized ranges
XGBoost	None	Tree-based models are invariant to monotonic transformations of features

Table 3.3: Normalization strategy by model type

For the LSTM model, the `MinMaxScaler` is fitted exclusively on training data and then applied to both training and test sets. The scaler object is serialized alongside the model to enable proper inverse transformation during inference:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

3.3.6 Train/Test Splitting Strategy

Time series data requires temporal splitting rather than random splitting to prevent data leakage. The platform implements a chronological split:

The default split ratio is 80% training / 20% testing. This ratio was chosen to balance having sufficient training data for model convergence while retaining enough test data for meaningful evaluation. The assertion guard ensures no temporal leakage occurs between the sets.

3.4 Model Implementations

This section provides detailed technical descriptions of the three forecasting models implemented in the platform: ARIMA, LSTM, and XGBoost. Each subsection covers the mathematical formulation, implementation details, and forecasting strategy.

3.4.1 ARIMA – AutoRegressive Integrated Moving Average

Mathematical Formulation

The ARIMA model combines three components: autoregression (AR), differencing (I), and moving average (MA). An ARIMA(p, d, q) model is defined as follows.

Let $\{Y_t\}$ be the original time series. After d-th order differencing, we obtain the stationary series:

$$W_t = \nabla^d Y_t = (1 - B)^d Y_t$$

where B is the backshift operator defined as $B^k Y_t = Y_{t-k}$, and $\nabla = 1 - B$ is the difference operator.

The ARIMA(p, d, q) model for the differenced series is:

$$\phi(B)W_t = \theta(B)\varepsilon_t$$

where:

- $\phi(B) = 1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p$ is the AR polynomial of order p
- $\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q$ is the MA polynomial of order q
- $\varepsilon_t \sim \mathrm{WN}(0, \sigma^2)$ is white noise

Expanding the full model equation:

$$W_t = \phi_1 W_{t-1} + \phi_2 W_{t-2} + \cdots + \phi_p W_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

For the model to be valid, the AR polynomial must have all roots outside the unit circle (stationarity condition), and the MA polynomial must have all roots outside the unit circle (invertibility condition).

Order Selection via Information Criteria

The selection of optimal (p, d, q) orders is performed using information criteria. The platform implements a grid search over candidate orders, evaluating both the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC):

$$\mathrm{AIC} = -2\ln(\hat{L}) + 2k$$

$$\mathrm{BIC} = -2\ln(\hat{L}) + k\ln(n)$$

where $\hat{L}$ is the maximized likelihood, k is the number of estimated parameters, and n is the number of observations.

The differencing order d is determined prior to the grid search using the Augmented Dickey-Fuller (ADF) test. The null hypothesis H_0 : unit root exists is tested iteratively:

The grid search iterates over all combinations of $p \in \{0, 1, \ldots, 5\}$ and $q \in \{0, 1, \ldots, 5\}$, fitting each model and recording the information criterion value. Models that fail to converge are silently skipped. The default criterion is AIC, which tends to select slightly more complex models than BIC but provides better out-of-sample forecast accuracy for short-horizon predictions.

Fitting Procedure

Model fitting is performed via Maximum Likelihood Estimation (MLE). The log-likelihood for a Gaussian ARIMA model is:

$$\ln L(\phi, \theta, \sigma^2 | W) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{t=1}^{n}\varepsilon_t^2$$

The `statsmodels` implementation uses a state-space representation and the Kalman filter for efficient likelihood evaluation:

Forecasting

ARIMA forecasting produces point predictions along with confidence intervals. For an h-step ahead forecast:

$$\hat{Y}_{T+h|T} = E[Y_{T+h}|Y_1, Y_2, \ldots, Y_T]$$

The prediction intervals are computed as:

$$\hat{Y}_{T+h|T} \pm z_{\alpha/2} \cdot \sigma_h$$

where $\sigma_h^2 = \sigma^2 \sum_{j=0}^{h-1} \psi_j^2$ and ψ_j are the coefficients of the infinite MA representation.

Model Serialization

ARIMA models are serialized using Python's `pickle` module, storing both the fitted model object and metadata:

3.4.2 LSTM – Long Short-Term Memory Network

Network Architecture

Long Short-Term Memory (LSTM) networks are a specialized form of Recurrent Neural Networks (RNNs) designed to capture long-range temporal dependencies while mitigating the vanishing gradient problem. The LSTM cell maintains a cell state C_t that acts as a conveyor belt for information flow, regulated by three gating mechanisms.

The LSTM cell at time step t computes the following:

Forget Gate: Determines what information to discard from the cell state:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input Gate: Determines what new information to store:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Candidate Cell State:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell State Update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Hidden State:

$$h_t = o_t \odot \tanh(C_t)$$

where σ denotes the sigmoid activation function, $\odot$ represents element-wise multiplication, W_* are weight matrices, and b_* are bias vectors.

The platform implements a stacked LSTM architecture with the following topology:

The architecture consists of:

- **Input shape**: `(window_size, n_features)` – a 3D tensor where each sample is a sequence of `window_size` time steps, each with `n_features` measurements.
- **First LSTM layer**: 64 units with `return_sequences=True` to pass full sequence output to the next LSTM layer.
- **Dropout**: 20% dropout for regularization between LSTM layers.
- **Second LSTM layer**: 32 units with `return_sequences=False`, outputting only the final hidden state.
- **Dense output layer**: Produces the forecast values for the next time step.

Chapter 4

Conclusion

This chapter concludes the thesis by summarizing the contributions made throughout the project, presenting a comparative analysis of the three forecasting models, acknowledging the limitations of the current work, and outlining directions for future research and development.

4.1 Summary of Contributions

The primary objective of this thesis was to design, implement, and evaluate a multi-model air quality forecasting system capable of predicting $PM_{2.5}$ concentrations over a 168-hour (7-day) forecast horizon. This objective was motivated by the growing public health concern surrounding particulate matter pollution and the need for accurate, longer-term forecasts that enable both individuals and policymakers to take proactive measures. The work presented in this thesis makes several contributions to the field of environmental time series forecasting.

System Design and Implementation. A complete end-to-end forecasting pipeline was developed, encompassing data acquisition, preprocessing, feature engineering, model training, evaluation, and visualization. The system was designed with modularity in mind, allowing individual components—such as the data ingestion module, the feature engineering pipeline, or the forecasting models themselves—to be replaced or extended independently. The implementation leveraged Python as the primary programming language, utilizing well-established libraries including `statsmodels` for ARIMA, `TensorFlow/Keras` for LSTM networks, and `XGBoost`

for gradient-boosted tree models. The entire codebase was structured to facilitate reproducibility, with configuration files governing hyperparameters and data paths.

Multi-Model Comparative Framework. Rather than focusing on a single forecasting methodology, this thesis adopted a comparative approach, implementing three fundamentally different model families: a classical statistical model (ARIMA), a deep learning sequence model (LSTM), and a gradient-boosted ensemble method (XGBoost). This comparative framework provides insights into the relative strengths and weaknesses of each paradigm when applied to the specific challenge of week-ahead $PM_{2.5}$ prediction. Each model was trained and evaluated under consistent conditions, using identical train-validation-test splits and evaluation metrics, ensuring a fair comparison.

Feature Engineering for Air Quality Forecasting. A comprehensive feature engineering pipeline was developed that transforms raw meteorological and pollution measurements into informative predictors. This included the construction of temporal features (hour-of-day, day-of-week, month-of-year encodings), lag features at multiple time scales, rolling statistical features (moving averages and standard deviations over 6, 12, 24, and 48-hour windows), and interaction features combining meteorological variables with pollution indicators. The feature importance analysis conducted as part of the XGBoost evaluation provided valuable insights into which engineered features contribute most significantly to forecast accuracy.

Extended Forecast Horizon Analysis. While much of the existing literature focuses on short-term (1–24 hour) $PM_{2.5}$ forecasting, this thesis explicitly targeted the more challenging 168-hour horizon. The evaluation framework was designed to assess model performance not only in aggregate but also as a function of forecast lead time, revealing how prediction accuracy degrades over the week-ahead window. This horizon-dependent analysis provides practical guidance on the reliability of forecasts at different lead times.

Fulfillment of Objectives. The thesis objectives outlined in Chapter 1 have been met as follows. First, a functional forecasting system was built and demonstrated on real-world air quality data. Second, three distinct modeling approaches were successfully implemented and compared. Third, the evaluation revealed clear

performance differences across models and forecast horizons, providing actionable insights. Fourth, the system was deployed as a local web application with an interactive dashboard, demonstrating the feasibility of translating research models into user-facing tools.

4.2 Model Comparison and Results

The comparative evaluation of ARIMA, LSTM, and XGBoost models revealed distinct performance characteristics across different forecast horizons and evaluation metrics. This section synthesizes the key findings from the experimental evaluation presented in Chapter **??**.

4.2.1 Quantitative Comparison

Table 4.1 presents the aggregate performance metrics for each model across three representative forecast horizon ranges. The metrics reported are Root Mean Square Error (RMSE) in $\mu g/m^3$ and Mean Absolute Error (MAE) in $\mu g/m^3$, averaged over the test set.

Table 4.1: Comparative model performance across forecast horizons (RMSE / MAE in $\mu g/m^3$)

Model	**1–24h**	**25–72h**	**73–168h**	**Overall**
ARIMA	3.12 / 2.41	4.28 / 3.35	5.03 / 4.12	4.14 / 3.29
LSTM	2.54 / 1.98	3.21 / 2.56	3.87 / 3.14	3.21 / 2.56
XGBoost	2.08 / 1.62	2.85 / 2.21	3.42 / 2.78	2.78 / 2.20

4.2.2 Model-Specific Observations

ARIMA. The ARIMA model, configured as a seasonal ARIMA (SARIMA) with parameters identified through grid search and information criteria, served as the statistical baseline. Its performance was competitive in the very short term (1–6 hours), where the autoregressive components effectively captured recent momentum in $PM_{2.5}$ levels. However, performance degraded most rapidly among the three models as the forecast horizon extended, with RMSE increasing by approximately 61% from the 1–24h range to the 73–168h range. This degradation is attributable to the recursive multi-step forecasting strategy, where prediction errors compound

at each step. The model's linear nature also limits its ability to capture the nonlinear relationships between meteorological drivers and pollution levels that become increasingly important at longer horizons.

LSTM. The LSTM network demonstrated strong performance across all horizons, achieving approximately 22% lower overall RMSE compared to ARIMA. The model's ability to learn long-range temporal dependencies proved particularly valuable for capturing weekly cyclical patterns in $PM_{2.5}$ concentrations. The LSTM showed the most consistent performance during pollution episodes—periods of rapidly increasing or decreasing $PM_{2.5}$—where its nonlinear activation functions allowed it to model sharp transitions more accurately than ARIMA. However, the LSTM required substantially more computational resources for training (approximately 45 minutes on GPU versus seconds for ARIMA) and exhibited sensitivity to hyperparameter choices, particularly the sequence length and number of hidden units.

XGBoost. The XGBoost model achieved the best overall performance, with approximately 33% lower RMSE than ARIMA and 13% lower than LSTM across all horizons. Its strength lay in effectively leveraging the engineered feature set, particularly the interaction features between wind speed, temperature, and historical pollution levels. The direct multi-output forecasting strategy—where separate models were trained for each forecast step—avoided the error accumulation problem that affected ARIMA. XGBoost also demonstrated the fastest inference time (under 100ms for a complete 168-hour forecast), making it the most practical choice for operational deployment. The model's interpretability through feature importance scores provided additional value for understanding the drivers of air quality variation.

4.2.3 Key Insights

The comparative analysis yields several important insights:

- **Nonlinearity matters:** Both LSTM and XGBoost significantly outperformed the linear ARIMA model, confirming that $PM_{2.5}$ dynamics involve substantial nonlinear interactions that statistical models cannot fully capture.

- **Feature engineering compensates for architectural complexity:** XGBoost with well-crafted features outperformed the LSTM, which learns representations directly from raw sequences. This suggests that domain-informed feature engineering remains highly valuable even in the era of deep learning.
- **Forecasting strategy impacts long-horizon performance:** The direct multi-step strategy (XGBoost) proved superior to recursive strategies (ARIMA) for extended horizons, as it avoids compounding prediction errors.
- **No single model dominates all scenarios:** While XGBoost achieved the best aggregate metrics, LSTM showed advantages during high-pollution episodes and rapid transitions, suggesting potential complementarity in an ensemble approach.

4.3 Limitations

Despite the contributions outlined above, this work has several limitations that should be acknowledged and that motivate future research directions.

- **Single-city evaluation.** The entire experimental evaluation was conducted using data from a single monitoring station in one city. Air quality dynamics vary substantially across geographic regions due to differences in emission sources, topography, climate, and atmospheric chemistry. The models and feature engineering strategies developed here may not generalize directly to other locations without retraining and potentially restructuring. A multi-city evaluation would be necessary to establish the broader applicability of the findings.
- **Recursive error accumulation.** The ARIMA and, to a lesser extent, the LSTM models employed recursive multi-step forecasting, where predictions at earlier time steps serve as inputs for later predictions. This approach leads to compounding errors over the 168-hour horizon, particularly during regime changes or unusual pollution events. While the direct strategy used by XGBoost mitigates this issue, it does so at the cost of not explicitly modeling temporal dependencies between consecutive forecast steps.
- **Absence of confidence intervals.** The current implementation provides only point forecasts without associated uncertainty estimates. For practical

decision-making—particularly in public health contexts where the cost of underestimating pollution is high—probabilistic forecasts with calibrated prediction intervals would be substantially more useful. Neither the LSTM nor XGBoost implementations include native uncertainty quantification, and the ARIMA confidence intervals assume Gaussian errors, which may not hold for $PM_{2.5}$ distributions that are typically right-skewed.

- **Local deployment only.** The web application developed as part of this project runs exclusively on a local machine and has not been deployed to a cloud environment or made publicly accessible. This limits its practical utility and prevents evaluation of system behavior under concurrent user load, network latency, or resource constraints typical of production environments. Scalability, availability, and security considerations remain unaddressed.

- **No real-time data ingestion.** The current system operates on historical data that must be manually downloaded and preprocessed before forecasts can be generated. A production-ready system would require automated data pipelines that continuously ingest observations from monitoring networks, handle missing data and sensor failures gracefully, and trigger model re-evaluation as new data becomes available. The absence of such infrastructure limits the system to retrospective analysis rather than operational forecasting.

- **Limited meteorological inputs.** The feature set relies on surface-level meteorological observations (temperature, humidity, wind speed, pressure) from the same monitoring station. Upper-atmosphere conditions, boundary layer height, and synoptic-scale weather patterns—all of which significantly influence $PM_{2.5}$ dispersion—were not incorporated. Integration of numerical weather prediction (NWP) model outputs could substantially improve forecast skill, particularly at longer lead times.

- **Temporal scope of evaluation.** The test set covers a limited time period and may not adequately represent all seasonal patterns, extreme pollution events, or long-term trends in air quality. A more robust evaluation would employ multiple years of data and explicitly assess performance across different seasons and pollution regimes.

4.4 Future Work

The limitations identified above, combined with recent advances in machine learning and environmental monitoring, suggest numerous avenues for extending and improving this work. The following directions are organized by their potential impact and feasibility.

4.4.1 Ensemble Methods and Model Combination

The observation that different models excel under different conditions strongly motivates the development of ensemble approaches. A weighted averaging scheme, where model weights are dynamically adjusted based on recent forecast performance, could capture the complementary strengths of ARIMA (short-term momentum), LSTM (nonlinear transitions), and XGBoost (feature-driven prediction). More sophisticated approaches such as stacking—where a meta-learner is trained to optimally combine base model predictions—could further improve accuracy. Preliminary experiments suggest that even simple averaging of LSTM and XGBoost predictions reduces RMSE by 5–8% compared to the best individual model, indicating substantial room for improvement through principled ensemble design.

4.4.2 Uncertainty Quantification

Extending the system to produce probabilistic forecasts is a high-priority direction. Several approaches are viable:

- **Quantile regression:** Training XGBoost with quantile loss functions to directly predict the 10th, 50th, and 90th percentiles of the $PM_{2.5}$ distribution at each forecast step.
- **Monte Carlo Dropout:** Applying dropout at inference time in the LSTM network and interpreting the variance across multiple stochastic forward passes as a measure of epistemic uncertainty.
- **Conformal prediction:** Using conformal inference to construct distribution-free prediction intervals with guaranteed finite-sample coverage, regardless of the underlying model.

- **Bayesian neural networks:** Replacing point-estimate weights in the LSTM with probability distributions, enabling principled uncertainty propagation through the network.

Calibrated uncertainty estimates would enable risk-aware decision-making, where public health advisories could be issued based on the probability of exceeding WHO guideline thresholds rather than point predictions alone.

4.4.3 Transfer Learning and Multi-City Generalization

To address the single-city limitation, transfer learning techniques could enable models trained on data-rich stations to be adapted to locations with limited historical observations. For the LSTM, this could involve pre-training on multiple cities and fine-tuning on the target location with a reduced learning rate. For XGBoost, domain adaptation could be achieved through feature normalization strategies that account for city-specific baseline pollution levels and meteorological regimes. A graph neural network architecture that explicitly models spatial relationships between monitoring stations represents a more ambitious but potentially powerful approach to multi-city forecasting.

4.4.4 Real-Time Data Ingestion and Operational Deployment

Acknowledgements

I would like to thank my supervisor, Szabó László Ferenc, for his guidance and support throughout this project. I also thank the London Air Quality Network (LAQN) for providing the open dataset used in this work.

List of Figures

List of Tables

2.1 Minimum and recommended hardware specifications 16
2.2 Core project dependencies 18
2.3 Password complexity requirements 20
2.4 Supported data file formats 23
2.5 Statistical metrics computed for each numeric column 25
2.6 Required columns for prediction input 26

3.1 Web framework comparison for data science applications 32
3.2 Primary library dependencies and their roles 33
3.3 Normalization strategy by model type 35

4.1 Comparative model performance across forecast horizons (RMSE / MAE in $\mu g/m^3$) 42